

5-1 · 분산 처리와 Spark 아키텍처 · 정리·회고 0.5H

정리·회고

개념 아홉 개를 한 장으로, 5-2 클러스터 준비 확인

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

- ① 개념 아홉 개를 「한 줄 정의 · UI 어디」로 정리한다
- ② 5-2 를 위해 Spark 이미지와 Docker 메모리를 확인한다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ 오늘의 체크리스트를 스스로 점검한다

cache 전후 시간과 파티션 수는 옆 사람과 비교해 보세요 — 환경마다 다르게 나옵니다

구조와 지연 평가 — 네 낱말

개념	한 줄 정의	UI 에서 어디
드라이버	계획을 세우고 결과를 모은다	Executors 탭의 driver
익스큐터	실제 계산을 한다	Executors 탭 (5-2 에서 늘어남)
transformation	계획만 쌓는다	Job 이 안 생김
action	실행을 시작한다	Job 이 하나 생김

드라이버는 계획, 익스큐터는 계산 · transformation 은 계획만, action 이 실행

UI 를 읽는 다섯 낱말

개념	한 줄 정의	UI 에서 어디
Job	action 하나당 하나	Jobs 탭
Stage	셔플을 경계로 나뉜다	Job 상세 · Stages 탭
Task	파티션 하나당 하나	Stage 상세
셔플	노드 간 데이터 재배포	계획의 Exchange
DAG	실행 계획 그림	Job 상세 DAG Visualization

Job(action) → Stage(셔플 경계) → Task(파티션) — 셔플은 계획의 Exchange 로 찾는다

다음 차시(5-2) 준비

셀을 나가고 이미지·메모리를 확인한다

셀 나가기	>>> exit()
이미지 확인	docker images grep spark
메모리 확인	docker info grep -i "total memory"
폴더	mkdir -p ~/data-eng/week5

확인: `apache/spark:3.5.3` 이미지가 있고 Docker 메모리 4GB 이상이면 준비 완료 — 8GB 이상 권장 · 5-2 는 컨테이너가 3개 이상 뜬다

여덟 개를 스스로 확인해 보세요

- 1 스케일업·스케일아웃 차이와 「한 대로 되면 한 대로」를 말할 수 있다
- 2 PySpark 셀을 컨테이너로 기동했다
- 3 드라이버/익스큐터의 역할을 비유 없이도 설명할 수 있다
- 4 transformation 단계에서 Job 이 안 생김을 UI 로 확인했다
- 5 action 뒤 Job → Stage → Task 계층을 UI 에서 열어 봤다
- 6 explain() 으로 계획을 읽고 Exchange(셔플)를 찾았다
- 7 groupBy 로 Stage 2개 · 파티션 수 = Task 수를 확인했다
- 8 cache 전후의 실행 시간을 직접 재 봤다

증상으로 찾는 원인과 해결

증상	원인	해결
컨테이너 종료 · OutOfMemoryError	Docker 메모리 부족 · 큰 데이터에 collect()	메모리 4GB 이상 · show().limit() 으로
localhost:4040 이 안 열린다	-p 4040:4040 누락 · action 을 한 번도 안 함	포트 매핑 후 재기동 · count() 뒤 새로고침
withColumn·filter 가 아무것도 안 나온다	오류가 아니라 지연 평가	show().count() 를 붙인다
셀을 나갔더니 4040 이 안 열린다	UI 는 앱이 살아 있는 동안만 뜬다	셀 재기동 = 새 UI · History Server(심화)

전체 데이터 확인은 collect() 대신 show() 나 limit() — 드라이버 한 곳으로 끌어오지 않는다

자주 나오는 질문 네 가지

- 1 Pandas 를 쓰면 안 되나요?**
메모리에 들어가면 Pandas 가 훨씬 빠르고 간단 — 작은 데이터에 Spark 는 오히려 느리다(분산 비용)
- 2 local[*] 인데도 분산인가요?**
코어 수만큼 나눠 병렬로 — 진짜 여러 대는 아니지만 구조가 같다, 5-2 에서 진짜 클러스터
- 3 cache 를 항상 쓰면 좋은 것 아닌가요?**
메모리를 차지한다 — 「두 번 이상 쓰는가」로 판단
- 4 Spark UI 는 실무에서 얼마나 쓰나요?**
아주 많이 — 느린 잡의 원인을 찾는 거의 유일한 도구, 오늘의 세 계층이 그 기초

더 해 볼 것 네 가지

- 1 같은 count() 두 번 + cache()**
UI 에서 두 Job 의 실행 시간을 비교하고, cache() 뒤 두 번째가 어떻게 달라지는지
- 2 기본 파티션 수의 가설**
df.rdd.getNumPartitions() 값이 어떻게 정해졌을지 — 답은 5-5 에서
- 3 spark.sql.shuffle.partitions = 8**
spark.conf.set 으로 바꾸고 groupBy 를 다시 — Stage 1 의 Task 수가 어떻게 달라지는지
- 4 join 하나의 explain()**
Exchange 가 몇 개 생기는지 세어 본다

다음 회차 예고

5-2 · Docker Compose 로 Spark 마스터-워커 구축

01

이론 1H

Spark 클러스터
배포 구조 이해

02

실습 1.5H

Compose 로 마스터-
워커 클러스터 구축

03

실습 1H

spark-submit
으로 잡 제출

04

정리 0.5H

정리·회고
마스터 UI 8080

준비: Docker Desktop + docker compose version 확인 · 5-1 단일 컨테이너는 종료 · 메모리가 부족하면 4주차 컨테이너를 내려 두기

5-1 · 정리·회고 0.5H 마무리

다음 회차는 5-2 Spark 마스터-워커 클러스터

compose 파일 하나로 마스터 1 + 워커를 띄우고, spark-submit 으로 잡을 제출해 마스터 UI 에서 추적합니다

✓ Standalone 마스터와 워커의 역할

✓ Compose 로 클러스터 기동 · 워커 등록 확인

✓ spark-submit 으로 애플리케이션 제출

✓ 마스터 UI(8080)에서 워커 상태 · 실행 이력

준비할 것

compose version 확인 · 5-1 셀 종료